Национальный исследовательский университет
«Высшая школа экономики»
Факультет компьютерных наук
Департамент программной инженерии

Многопоточная симуляция шлюза

Отчёт по индивидуальному заданию 2
Архитектура компьютера и операционные системы
Вариант 47

Выполнил: Турицын Матвей Максимович
Группа: БПИ-252

Москва 2026

1 Постановка задачи

Разработана консольная многопоточная программа на языке C для Linux с использованием pthread. Программа моделирует проход людей через шлюз космической станции, согласует действия участников и оборудования, учитывает ресурсы и сохраняет события в файл журнала.

Программа развивает итоговую версию первого домашнего задания. Из неё без изменений перенесён airlock.c с функциями дверей, давления, перемещения людей, проверки безопасности и расчёта ресурсов. В структурах сохранены состояния и направления; вместо счётчиков тиков используются потоки и длительности в миллисекундах.

Условие варианта 47

Шлюз имеет внутреннюю и внешнюю двери, ограниченную вместимость и систему изменения давления. Люди подают заявки на выход в космос и возвращение на станцию. Внутреннюю дверь можно открыть при нормальном давлении, внешнюю — при низком. Одновременное открытие дверей запрещено. Аварийное возвращение получает приоритет в очереди. Необходимо учитывать время операций, запас воздуха и энергии и возможные отказы оборудования.

Сценарий прохода

При выходе в космос диспетчер подготавливает нормальное давление, открывает внутреннюю дверь и разрешает участникам войти. После закрытия внутренней двери давление снижается, открывается внешняя дверь и участники выходят. Проход заканчивается закрытием внешней двери. При возвращении используется обратная последовательность.

Начальное состояние: камера пустая, обе двери закрыты, давление нормальное. Диспетчер выбирает среди уже поступивших заявок сначала аварийные, затем более ранние. При равенстве параметров сохраняется порядок записей во входном файле. В группу включаются только ожидающие участники одного направления. Уже начатый проход не прерывается ради новой заявки.

Ресурсы и безопасность

Открытие или закрытие двери расходует 1 единицу энергии. Изменение давления расходует 3 единицы энергии; восстановление нормального давления дополнительно расходует 5 единиц воздуха. До посадки проверяется запас на полный проход выбранного направления, включая начальную подготовку давления.

Проверяются условия открытия дверей и изменения давления, вместимость камеры и наличие ресурсов. При моделируемом отказе выполняется аварийное закрытие двери. Аварийный привод считается исправным, но также расходует энергию. Люди могут остаться в закрытой камере; их положение выводится в итогах.

2 Потоки и синхронизация

Поток	Назначение
Главный поток	Диспетчер: выбор группы, проверка ресурсов, команды и ожидание результатов.
По одному на человека	Задержка до заявки, постановка в очередь, вход и выход по разрешению.
Внутренняя дверь	Открытие и закрытие по команде с заданной длительностью.
Внешняя дверь	Открытие и закрытие по команде с заданной длительностью.
Система давления	Изменение давления при закрытых дверях.

Всего создаётся N + 4 потока с учётом главного, где N это число людей. Для трёх участников работают семь потоков. Все моделируемые сущности с поведением имеют собственный поток; диспетчер реализован главным потоком.

Общие данные

Один pthread_mutex_t защищает состояния людей, двери, давление, число людей в камере, ресурсы, команды и признак завершения. Записи событий во время работы потоков также выполняются под этим мьютексом, поэтому строки разных участников не смешиваются. Итоговый вывод производится после завершения рабочих потоков.

Передача команд

У каждого человека и устройства есть два семафора: request для команды и finished для подтверждения выполнения. Диспетчер записывает команду под мьютексом, освобождает его, вызывает sem_post и ожидает finished. Исполнитель получает команду через sem_wait и после изменения состояния подтверждает завершение.

Семафор arrivals уведомляет диспетчер о новых заявках. Если очередь пуста, диспетчер использует ожидание с тайм-аутом, чтобы периодически проверять запрос завершения. Мьютекс не удерживается во время ожидания семафора или задержки оборудования. Поэтому люди могут подавать заявки, пока выполняется операция двери или давления.

Зависимые действия шлюза выполняются последовательно по правилам безопасности. При этом потоки существуют одновременно и взаимодействуют асинхронно. Состав группы зависит от того, какие заявки успели поступить к моменту выбора. Одинаковый seed не гарантирует одинаковый порядок событий и расход ресурсов при разных составах групп.

3 Входные данные и запуск

main.c управляет потоками и завершением; airlock.c содержит исходные правила шлюза; dispatcher.c содержит команды и обработчики потоков; config.c содержит чтение параметров; output.c это журнал. Структуры объявлены в person.h, airlock.h и dispatcher.h. Сборка задана в Makefile. В README.md описаны параметры. Программа принимает два аргумента: путь к входному файлу и путь к новому файлу журнала.

```
make
./airlock input.txt run.log
```

Формат входного файла

Первая строка содержит девять значений: вместимость, запас воздуха, запас энергии, длительность операции двери в мс, длительность изменения давления в мс, вероятность отказа в процентах, seed, режим групп и включение аварийного приоритета. Последний параметр: 1 — включить, 0 — отключить. Режим 1 включает группы, 0 — обслуживание по одному.

Далее указано число людей, затем по четыре значения на человека: уникальный положительный id, направление, задержка заявки в мс и аварийный приоритет. Направление 0 означает выход в космос, 1 — возвращение на станцию. Приоритет 1 допускается только при возвращении.

```
2 20 30 200 300 0 42 1 1
3
1 0 0 0
2 1 400 1
3 0 0 0
```

В примере вместимость равна двум, запас воздуха — 20, энергии — 30. Дверь выполняет операцию за 200 мс, система давления — за 300 мс. Отказы отключены. Участники 1 и 3 направляются в космос без начальной задержки; участник 2 запрашивает аварийное возвращение через 400 мс.

Число людей ограничено 64, вместимость — 16, как в первом задании. Длительности оборудования: 1–60000 мс; задержка заявки: 0–3600000 мс. Проверяются диапазоны, направления, приоритеты, уникальность id и лишние данные после последней записи. При неверном файле программа завершается с кодом 1.

Вывод и завершение

Каждая строка содержит время с начала запуска в формате [чч:мм:сс]. Выводятся заявки, состав групп, начало и завершение операций, перемещения людей, отказ или нехватка ресурсов и итоговые состояния. Для короткого запуска несколько событий могут иметь одинаковое время.

Нормальное завершение возвращает код 0. Ctrl+C и SIGTERM устанавливают флаг остановки. Диспетчер прекращает обслуживание, закрывает двери аварийным приводом, открывает ожидающие потоки и вызывает pthread_join. Семафоры и мьютекс уничтожаются, журнал закрывается. Остановка и ошибки возвращают код 1.

4 Результаты проверки

Проверка выполнена в Ubuntu WSL. Программа собрана GCC с флагами -Wall -Wextra -Wpedantic -Werror, -pthread и проверкой неопределённого поведения -fsanitize=undefined. В проверенных запусках предупреждений компилятора и сообщений санитайзера не получено.

Для проверок использовались временные копии входных данных. Отдельные тестовые наборы и скрипты в состав сдаваемого проекта не включены. Ниже приведены фактически полученные результаты; время поступления сигнала и планирование потоков могут менять промежуточные состояния.

Проверка	Полученный результат
Обычный запуск	Успешно 3 из 3. Камера пустая, двери закрыты. Воздух 15, энергия 9. Код 0.
Вероятность отказа 100%	Выполнено 0 из 3. Двери закрыты, камера пустая. Воздух 20, энергия 30. Код 1.
Начальная энергия 0	Остановка до посадки. Выполнено 0 из 3, камера пустая. Воздух 20, энергия 0. Код 1.
SIGINT и SIGTERM примерно через 250 мс	Выполнено 0 из 3, один человек остался в камере. Двери закрыты. Воздух 20, энергия 28. Код 1.

Пример итогового вывода

```
Person 1: space, done
 Person 2: station, done
 Person 3: space, done
 Result: success; completed=3/3; inside=0;
 air=15; energy=9; doors=0,0
```

В этом запуске участники проходили по одному, поскольку заявки стали видны диспетчеру в разные моменты. Режим групп не предусматривает дополнительного ожидания заполнения камеры. При совместном проходе участников расход энергии может быть меньше.

Использование ИИ

Для подготовки отчёта и README использовался ChatGPT. Весь текст отчёта был сформирован моделью на основе требований к заданию, исходного кода программы и результатов проведённых проверок.

Итог

Реализована многопоточная модель шлюза с очередью заявок, приоритетом аварийного возвращения, учётом ресурсов и отказов. Мьютекс защищает общие состояния, семафоры согласуют команды и результаты. Программа читает сценарий из файла, сохраняет журнал и корректно завершила проверенные штатные и аварийные сценарии. Оба сигнала проверены отдельными запусками. Дополнительно подтверждён отказ запуска при нулевой вместимости.